

Adaptive Smart Cabin & Vehicle Health Monitoring System

Architecture Explanation Document

C++17 · CMake · Multi-threaded · AUTOSAR-inspired · Hackathon v1.0

Confidential — Internal Team Document

Generated: May 2026

This document explains how the **Adaptive Smart Cabin & Vehicle Health Monitoring System** is structured. It is written for someone who wants to understand the codebase quickly — not just what the modules are called, but *why* they were designed that way, how data moves through the system, and how the advanced bonus features fit into the picture. No prior knowledge of the codebase is assumed.

#	Section	Page
1	What Is This Project?	2
2	File & Module Structure	2
3	The Four-Layer Architecture	3
4	How the Include Chain Works	3
5	Data Flow — Sensor to Screen	4
6	Thread Model & Concurrency	4
7	Runtime Object Ownership	5
8	Module-by-Module Reference	5
9	Bonus Features Deep-Dive	7
10	Alert Thresholds & Configuration	9
11	Compiled Binary Metrics	10
12	Automotive Mapping (AUTOSAR)	10

Table of Contents

1

What Is This Project?

The **Smart Cabin & Vehicle Health Monitoring System** is a real-time automotive simulation written in **C++17**. Think of it as the software inside a car's main ECU — it reads six different vehicle sensors every 500 ms, checks each reading against safety thresholds, decides whether to raise an alert, displays a live dashboard in the terminal, and writes everything to a persistent log file.

The project goes further than a basic sensor monitor. It also simulates systems you'd find in a real modern vehicle: a CAN bus communication layer, OBD-II diagnostic trouble codes, ECU health watchdogs,

over-the-air firmware updates, an event data recorder, and a service-oriented message bus. These are not afterthoughts — they are fully wired into the threading model and run in parallel with the core monitoring loop.

Sensor	Range	Warning Trigger	Critical Trigger
Engine Temperature	60 – 130 °C	—	> 110 °C
Battery Voltage	7 – 15 V	—	< 10 V
Speed	0 – 180 km/h	—	> 120 km/h
Tyre Pressure	15 – 40 PSI	< 25 PSI	—
Door Status	OPEN / CLOSED	—	Open + Speed > 10
Seatbelt Status	LOCKED / UNLOCKED	Unlocked + Moving	—

Table 1 — Six monitored vehicle sensors and their alert thresholds

2 File & Module Structure

Every module follows the standard C++ pattern: a **.hpp header file** declares the public interface (what a module can do), and a **.cpp source file** contains the implementation (how it does it). This split keeps compile times fast and lets each module be swapped or tested independently.

File	Lines	Role
include/Sensor.hpp + src/Sensor.cpp	~350	Abstract sensor base class and 6 concrete sensor types
include/Alert.hpp + src/Alert.cpp	~680	Alert class, AlertManager, AlertRule strategy pattern, Rule of Five
include/Logger.hpp (header-only template)	~200	EventLogger<T> template — RAII file I/O, async queue, lambda search
include/Dashboard.hpp + src/Dashboard.cpp	~520	VehicleStatistics, Dashboard renderer, sparkline graphs
include/DriverProfile.hpp	~20	DriverProfile struct with configurable thresholds per driver
include/DtcManager.hpp + src/DtcManager.cpp	~180	OBD-II DTC registry, FreezeFrame capture, Singleton
include/EcuMonitor.hpp + src/EcuMonitor.cpp	~130	ECU health telemetry (CPU/mem/rx) for 5 named ECUs
include/EdrRecorder.hpp + src/EdrRecorder.cpp	~140	Crash-safe Event Data Recorder with circular buffer
include/OtaSimulator.hpp + src/OtaSimulator.cpp	~100	OTA firmware update state machine (7 states)
include/CanBus.hpp + src/CanBus.cpp	~140	CAN frame encode/decode, bus load tracking, error injection
include/ServiceBus.hpp + src/ServiceBus.cpp	~140	Pub/sub service bus — SensorEvent, AlertEvent, DtcEvent
include/JsonParser.hpp (header-only)	~200	Custom JSON parser used to load driver_profiles.json
src/ThreadManager.cpp	~430	Spawns and joins 5 worker threads, manages lifecycle
src/main.cpp	~1050	Composition root: creates objects, wires deps, interactive menu
src/tests.cpp	~600	Automated test suite — constructors, rules, thread safety
data/config.txt	—	INI-style config: intervals, thresholds, paths
data/driver_profiles.json	—	JSON file: 3 driver profiles loaded at startup

File	Lines	Role
data/test_cases.json	—	JSON file: test case definitions for automated tests

Table 2 — Complete file reference with approximate line counts

The system is organised in four **layers**, stacked from foundation to output. The rule is simple: *a layer may only use modules from the layers below it*. Nothing reaches upward. This prevents the tangled cross-dependencies that make large C++ projects hard to change. The structure mirrors Adaptive AUTOSAR's layering — which is intentional, because the project is designed to demonstrate automotive software thinking.

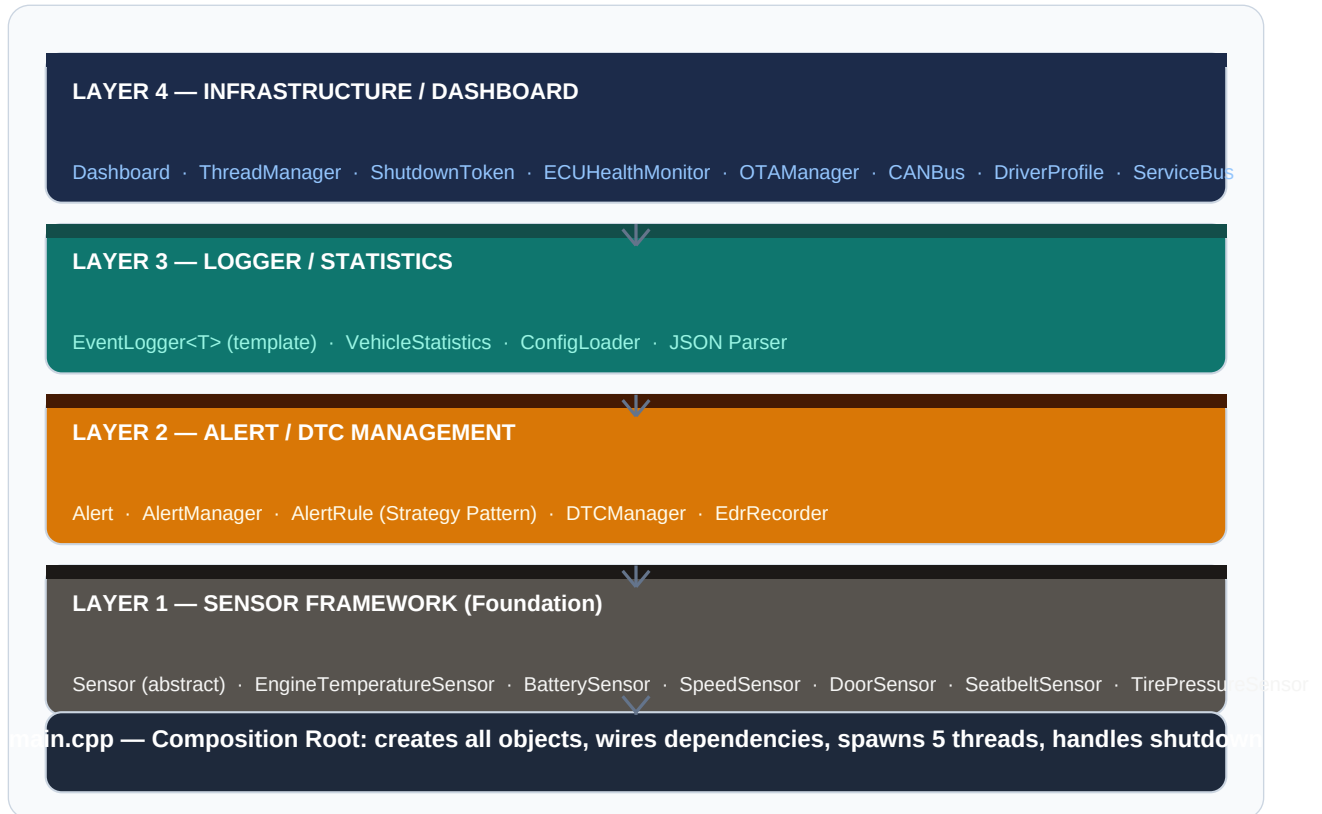


Figure 1 — Four-layer architecture. Arrows show 'uses' direction (downward only).

Layer	Header(s)	What it does	Uses
4 — Dashboard / Infrastructure	Dashboard.hpp	Renders the terminal UI; owns all threads; contains ECUHealthMonitor, OTAManager, CAN bus, DriverProfile, ServiceBus	Layers 1, 2, 3
3 — Logger / Statistics	Logger.hpp	RAII file logging (EventLogger<T> template); rolling statistics; INI config loader	Layer 2
2 — Alert / DTC	Alert.hpp	Evaluates sensors against thresholds; manages active alerts (frequency); registers DTCs; stores them	Layer 1
1 — Sensor Framework	Sensor.hpp	Abstract Sensor base class; six concrete sensor simulations; per-object RNG; thread-local	None

Table 3 — Layer responsibilities and dependencies

The C++ compiler needs to know the definition of a class before it can use it. This means headers must be included in the right order — and no header can include another header that eventually loops back to include itself (circular dependency). The project handles this cleanly with a one-way include chain and a strategic forward declaration:

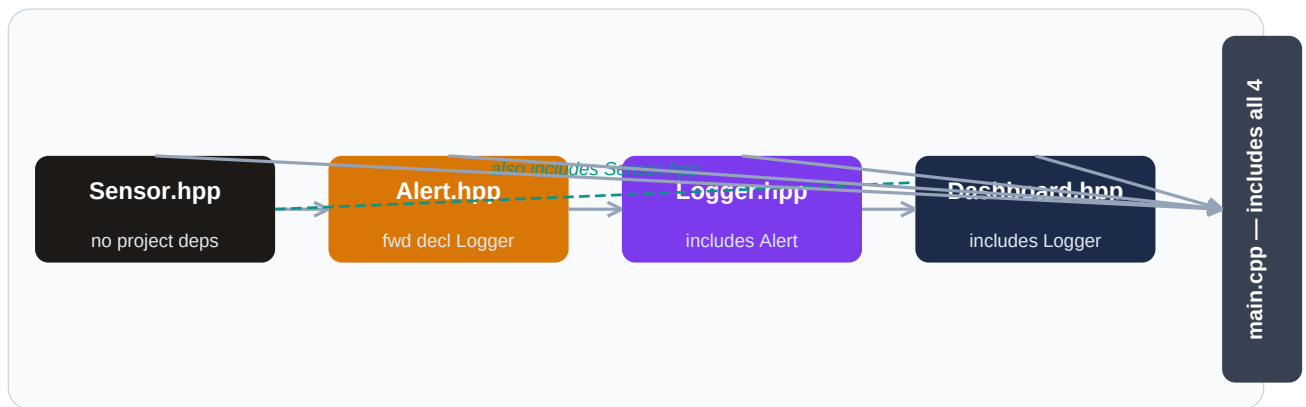


Figure 2 — Header include chain. One direction, no cycles.

Forward Declaration — Why It Exists

Alert.hpp needs to hold a pointer to a **Logger** for writing alerts to the log file. But **Logger.hpp** includes **Alert.hpp** — so **Alert.hpp** cannot include **Logger.hpp** back without creating a loop. The solution: **Alert.hpp** uses a forward declaration 'class EventLogger;' which just tells the compiler the class exists. The actual `#include Logger.hpp` only appears in **Alert.cpp** where **Logger**'s functions are called. This is a core C++ technique for breaking circular dependencies.

5 Data Flow — Sensor to Screen

Here is the complete path that data travels — from the moment a sensor generates a new reading to the moment it appears on screen and in the log file:

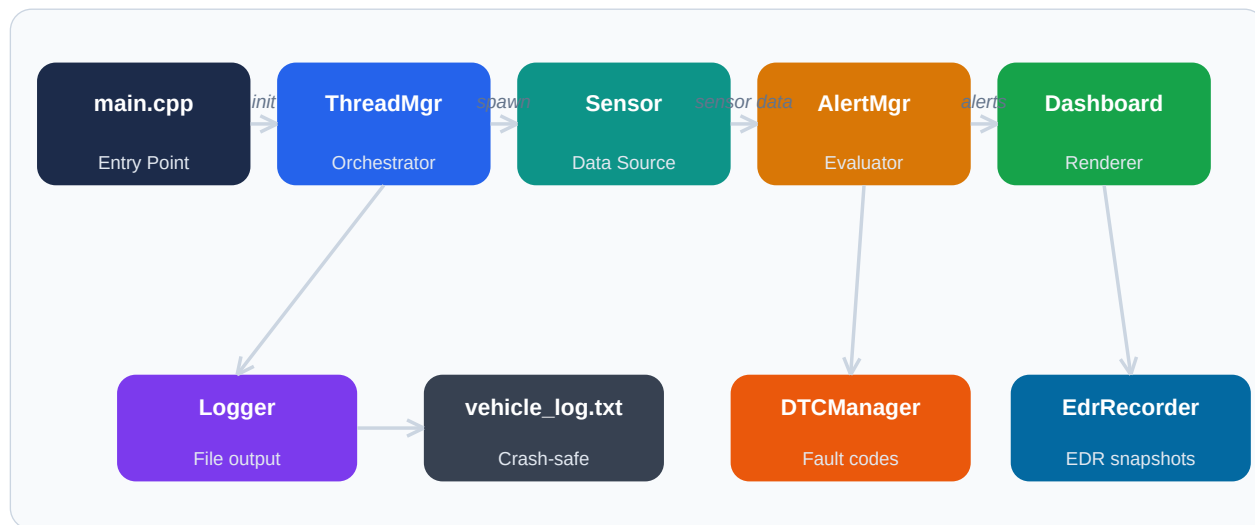


Figure 3 — Complete data flow from sensor update to dashboard render and file output.

Step	Thread	Module(s)	What Happens
1	Sensor (T1)	Sensor.cpp	update() called on all 6 sensors. Each generates a new reading using its Mersenne
2	Monitor (T2)	Alert.cpp, DTCManager	Reads all sensor values under mutex. Compares against active driver profile thresh
3	Alert dedup	AlertManager	Before adding an alert, checks if the same source is already active (O(1) array loo
4	DTC	DtcManager.cpp	If alert severity is CRITICAL, triggerDtc() registers an OBD-II fault code (e.g. P021
5	EDR	EdrRecorder.cpp	Every monitor cycle, recordTick() logs a snapshot into a 20-entry circular buffer. If
6	Logger (T4)	Logger.hpp (template)	All log calls go through EventLogger<string>::log() which is mutex-protected. File i
7	Dashboard (T3)	Dashboard.cpp	Every 2000 ms, display() renders header, all 6 sensor readings, active alerts sorte
8	ServiceBus	ServiceBus.hpp	After each alert change, monitor thread publishes an AlertEvent to all subscribers.

Table 4 — Step-by-step data flow through all threads and modules

6 Thread Model & Concurrency

The system runs **four worker threads** plus the main thread (which handles the interactive menu). All threads are spawned by **ThreadManager** and share a **ShutdownToken** — an atomic boolean with a condition_variable — so that when the user quits, every thread wakes up immediately and exits cleanly without polling or busy-waiting.

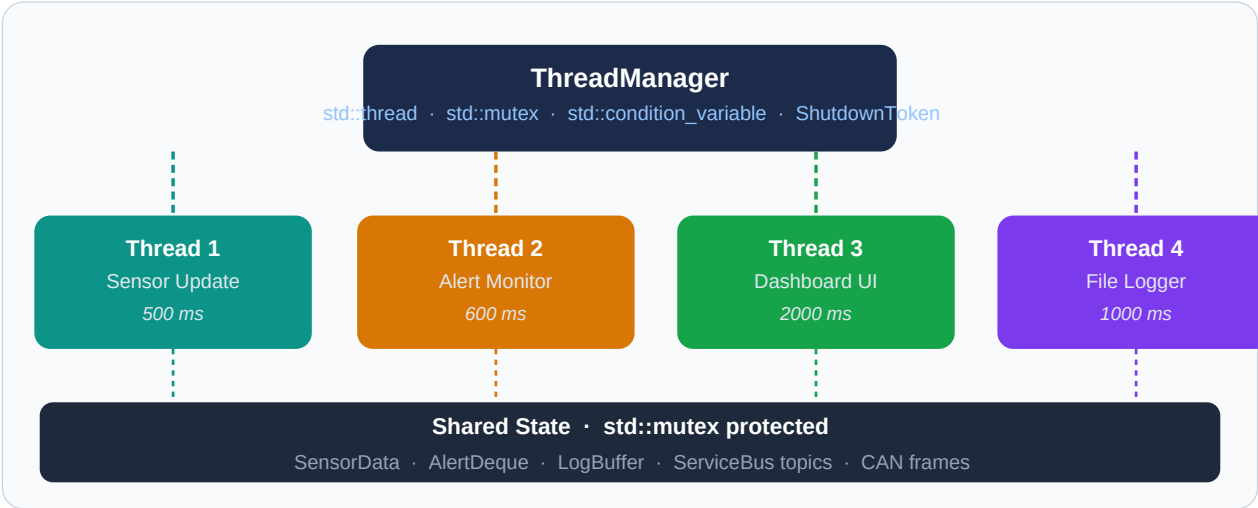


Figure 4 — Thread model. Dashed lines show access to shared state; all accesses are mutex-protected.

Thread	Update Rate	Sync Mechanism	Responsibility
T1 — Sensor Update	Every 500 ms	mutex on SensorData	Calls update() on all 6 sensors; generates new readings
T2 — Alert Monitor	Every 600 ms	condition_variable + mutex	Reads sensors, evaluates rules, raises/clears alerts, trig
T3 — Dashboard UI	Every 2000 ms	read lock on sensors and alerts	Renders full terminal dashboard including graphs and O
T4 — File Logger	Every 1000 ms	mutex on file handle + log queue	Drains async log queue to vehicle_log.txt with flush after
Main — Interactive	On demand	ShutdownToken broadcast	Handles user input: switch profile, view DTCs, trigger O

Table 5 — Thread responsibilities and synchronisation mechanisms

Shutdown Protocol — How Threads Stop Cleanly

The ShutdownToken contains an atomic<bool> and a condition_variable. When the user selects quit, the main thread sets the flag and calls notify_all(). Every worker thread is sleeping on condition_variable::wait_for — they all wake up instantly, check the flag, and exit their loops. ThreadManager::~ThreadManager() then joins all threads before destruction. No thread is ever killed forcefully.

The project uses no raw new or delete. Every heap object is managed by a smart pointer. This follows **RAII** — the object's lifetime is tied to its owner; when the owner is destroyed, the object is automatically cleaned up. There are two ownership patterns in this codebase:

Pointer Type	What it means	Used For
shared_ptr<T>	Multiple modules share ownership. Object lives until the last shared_ptr to it is destroyed.	Logger, AlertManager, STOManager, VehicleStatistics — shared across modules
unique_ptr<Sensor>	Exactly one owner. Cannot be copied, only moved. Enforces exclusive ownership.	Each concrete Sensor object stored in vector<unique_ptr<Sensor>>
unique_ptr<AlertRule>	Each rule strategy is owned exclusively by AlertManager.	EngineTempRule, BatteryVoltageRule, SpeedRule, TirePressureRule

Table 6 — Smart pointer ownership model

The sensor list deserves special mention. Sensors are stored as `vector<unique_ptr<Sensor>>` — a vector of pointers to the *abstract base class*. When the sensor thread calls `s->update()`, C++ automatically calls the correct overriding version for each concrete sensor type (`EngineTemperatureSensor`, `BatterySensor`, etc.) without the caller needing to know which specific type it is. This is **runtime polymorphism** in practice.

Sensor Module (Sensor.hpp / Sensor.cpp)

Foundation of the entire system. Defines the abstract **Sensor** base class with four pure virtual functions: `update()`, `display()`, `getValue()`, `getValueString()`. Six concrete classes inherit from it, each owning a per-object `std::mt19937` Mersenne Twister RNG seeded from `std::random_device` — so simulated readings are statistically independent. A static counter `sensorCount_` tracks how many Sensor objects are alive at any given moment (RAII increment/decrement in constructor/destructor). Sensors also support manual `setValue()` overrides for testing purposes.

Alert Module (Alert.hpp / Alert.cpp)

Two distinct components live here. The **Alert** class is a single event record — it stores severity (INFO through CRITICAL), message, source sensor name, timestamp, and a dynamic priority score. It implements the full **Rule of Five**: copy constructor, move constructor, copy assignment, move assignment, and destructor. Operators `<` and `==` are overloaded for priority sorting and deduplication. The **AlertManager** owns the active alert deque (bounded at 50 entries) and an unbounded history vector. It runs six pluggable **AlertRule** strategy objects — one per sensor type — making threshold logic extensible without modifying AlertManager itself. Lambda-based `filterAlerts()` lets callers query alerts with arbitrary predicates.

Logger Module (Logger.hpp — header-only template)

The most architecturally interesting module. **EventLogger<T>** is a C++ template class that logs any type `T` that supports operator<<. It opens its log file in the constructor and writes a footer + closes it in the destructor — pure RAII. Copy is deleted (file handles cannot be duplicated); move is allowed. Every write goes through a `std::mutex`. The file is `flush()`-ed after every write, ensuring no data is lost if the process crashes. An async queue (`logQueue_`) lets producer threads call `enqueue()` without blocking on file I/O; the logger thread calls `processQueue()` to drain it. An in-memory history vector supports lambda-based search() via `std::copy_if`.

Dashboard Module (Dashboard.hpp / Dashboard.cpp)

Handles visualisation and statistics. **VehicleStatistics** maintains an `std::map<string, SensorStats>` tracking min, max, average, and count per sensor — updated every monitor cycle. The **Dashboard** class is a stateless renderer (one mutable display counter aside) that writes formatted ASCII output to any `std::ostream` — it can target `std::cout`, a file, or a string stream for unit testing. It renders: system header, all sensor readings with status colour codes, active alerts sorted by priority, statistics summary, a sparkline graph for speed history, ECU health table, DTC list, and OTA progress bar.

ThreadManager (src/ThreadManager.cpp)

The conductor. Creates four `std::thread` objects and holds them in a vector. Each thread is given a lambda capturing references to all the modules it needs. `ThreadManager` also holds the `ShutdownToken` (`atomic bool + condition_variable`). On destruction, it sets the shutdown flag, calls `notify_all()`, and joins every thread in order. This guarantees that no thread outlives the data it references.

The ten bonus features are not isolated add-ons — each is a fully integrated module that interacts with the core threading model and data flow. Here is how each one works inside the codebase:

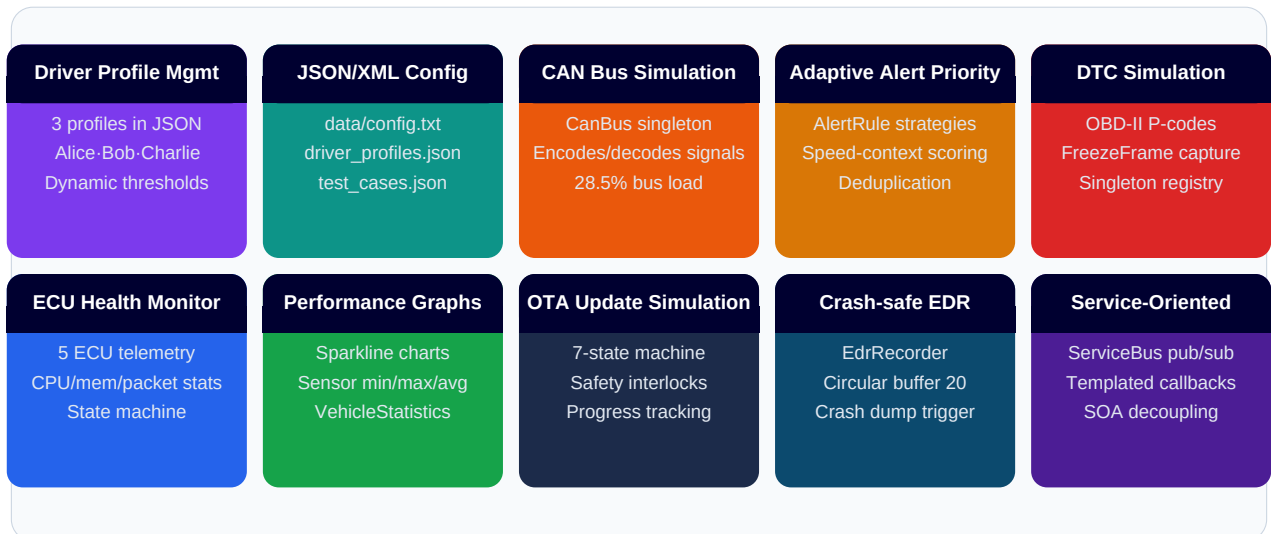


Figure 5 — Ten bonus features at a glance.

1. Driver Profile Management

Three driver profiles are defined in `data/driver_profiles.json`: **Alice (Default)** — standard thresholds; **Bob (Eco)** — conservative thresholds (engine temp warning at 105 °C, max speed 100 km/h); **Charlie (Valet)** — strict limits (max speed 80 km/h). Each profile is a `DriverProfile` struct loaded at startup by the JSON parser. The active profile is passed to `AlertManager::setActiveProfile()` and its threshold fields replace the compile-time constants in the alert evaluation loop. The interactive menu lets the user switch profiles at runtime.

2. JSON / XML Configuration Loading

Two separate loading mechanisms coexist. `data/config.txt` is a simple key=value INI file parsed by the `ConfigLoader` class inside `Logger.hpp` — it sets sensor update intervals, alert thresholds, log file path, and simulation settings. `data/driver_profiles.json` is a full JSON array parsed by the header-only `JsonParser.hpp` — a hand-written recursive descent parser that handles strings, numbers, booleans, arrays, and nested objects without any third-party libraries. A third file, `data/test_cases.json`, defines structured test scenarios loaded by the automated test suite.

3. CAN Bus Simulation

The `CanBus` singleton (`include/CanBus.hpp`) models a Controller Area Network frame bus. Every sensor reading can be encoded into a `CanFrame` struct (11-bit ID, 8-byte payload) using `encodeSignal(physicalValue, scale, offset)`, and decoded back with `decodeSignal()`. The bus tracks TX/RX frame counts, simulates a 28.5% nominal bus load, and supports `injectBusError()` for fault testing. In the main monitor loop, sensor values are periodically transmitted as CAN frames and logged — mirroring what a real ECU would do on the vehicle network.

4. Adaptive Alert Prioritization

AlertManager uses the **Strategy Pattern** via the AlertRule abstract base class. Each rule (EngineTempRule, BatteryVoltageRule, SpeedRule, etc.) is registered in a `vector<unique_ptr<AlertRule>>` and evaluated every monitor cycle. After evaluation, `prioritizeAlerts(speedKmh)` recomputes dynamic priority scores: a door-open alert scores higher at 90 km/h than at 5 km/h. The active deque is then re-sorted so the dashboard always shows the most urgent alert first. The C++17 pre-indexed `activeAlertsByType_[6]` bool array allows O(1) duplicate checking.

5. Diagnostic Trouble Code (DTC) Simulation

DtcManager is an OBD-II compliant singleton. When the alert monitor detects a CRITICAL condition, it calls `triggerDtc(code, description, sensorValues...)` which registers a DiagnosticTroubleCode entry in a `std::map<string, DTC>`. The first occurrence sets status to PENDING; a second occurrence confirms it. Each DTC captures a FreezeFrame — a full snapshot of all six sensor values at the exact moment of fault. The `executeObdService(serviceId)` method emulates real OBD-II Mode 03 (read DTCs) and Mode 04 (clear DTCs) commands.

6. ECU Health Monitor

EcuMonitor (EcuMonitor.hpp) tracks five named ECUs: BodyControlModule, EngineControlUnit, TransmissionECU, BrakingSystemECU, ADAS_Controller. Each has an EcuHealth struct with CPU load (%), memory allocated (KB), RX packet success rate (%), and current state (ONLINE/STANDBY/OFFLINE/BOOTING/UPDATING). The `updateTelemetry(tick)` method is called every monitor cycle and applies small random fluctuations to simulate realistic ECU telemetry. The Dashboard's `displayEcuStatus()` renders this as a formatted table. If an ECU transitions to OFFLINE, an alert is raised.

7. Performance Graph Statistics

VehicleStatistics in Dashboard.hpp maintains a `std::map<string, SensorStats>` — one entry per sensor name. SensorStats tracks min, max, cumulative sum, and sample count, with an `average()` method. Every monitor cycle calls `update(name, value)` for each sensor. The Dashboard renders this as a formatted table showing min/max/avg for each sensor. Speed values are also kept in a `vector<double>` `speedHistory` which feeds `displaySparklineGraph()` — a bar chart rendered using ASCII block characters directly in the terminal.

8. OTA Update Simulation

OtaSimulator is a Singleton with a 7-state atomic state machine: IDLE → DOWNLOADING → VERIFYING → FLASHING → REBOOTING → COMPLETED (or FAILED). The user triggers it from the interactive menu. Before starting, `startUpdate()` checks two safety interlocks: **speed must be 0 km/h** and **no active DTCs**. If either condition fails, the update is blocked and an error message is returned. Once started, `tick()` advances the state machine one step per call, incrementing a progress percentage. The Dashboard's `displayOtaStatus()` renders the current state and progress bar.

9. Crash-Safe Event Recording (EDR)

EdrRecorder maintains a `std::deque<EdrRecord>` capped at 20 entries (representing 10 seconds at 500 ms ticks). Every monitor cycle calls `recordTick()` with the current sensor snapshot including a simulated G-force reading. The deque automatically drops the oldest entry when full — giving a rolling 10-second window. When the monitor detects a crash condition (sudden extreme deceleration G-force), it calls `triggerCrashDump(reason)` which logs the entire pre-event window to a separate crash log file. This mirrors an automotive black box.

10. Service-Oriented Communication (ServiceBus)

ServiceBus is a templated publish-subscribe message bus. Modules communicate through typed event topics without depending on each other directly. Three event types are defined: `SensorEvent` (name, value, unit, timestamp), `AlertEvent` (source, message, severity, isActive), and `DtcEvent` (code, description, isConfirmed). Modules subscribe with a lambda callback; publishers call `publish(topic, event)` without knowing who is listening. All access is protected with a `shared_mutex` — read operations (publish dispatching) take shared locks; write operations (subscribe registration) take exclusive locks. This mirrors Adaptive AUTOSAR's `ara::com SOME/IP` service layer.

10 Alert Thresholds & Configuration

Thresholds are defined in two places: compile-time constexpr constants in AlertManager (zero runtime cost) and per-driver JSON values that override them at runtime. The table below shows the defaults and per-profile overrides:

Parameter	Default (Alice)	Bob (Eco)	Charlie (Valet)	Source
Max Speed	120 km/h	100 km/h	80 km/h	driver_profiles.json
Engine Temp Critical	110 °C	105 °C	100 °C	driver_profiles.json
Battery Voltage Low	10.0 V	10.5 V	11.0 V	driver_profiles.json
Tyre Pressure Low	25 PSI	25 PSI	25 PSI	constexpr (fixed)
Door Open (speed)	10 km/h	10 km/h	10 km/h	constexpr (fixed)
Sensor Update Interval	500 ms	500 ms	500 ms	config.txt
Dashboard Refresh	2000 ms	2000 ms	2000 ms	config.txt
Max Active Alerts	50	50	50	config.txt

Table 7 — Alert thresholds by driver profile

DTC Code	Fault Description	Trigger Condition	Captures FreezeFrame
P0217	Engine Coolant Overtemp	Temp > engineTempThreshold	Yes
P0563	System Voltage High	Voltage > 15 V	Yes
P0562	System Voltage Low	Voltage < 10 V	Yes
C0036	TPMS Pressure Low	Pressure < 25 PSI	Yes
B1001	Door Ajar While Moving	Door open + speed > 10	Yes
B1010	Seatbelt Reminder Active	Seatbelt unlocked + moving	Yes
U0100	Speed Sensor Comm Fault	Speed > 120 km/h	Yes

Table 8 — OBD-II Diagnostic Trouble Codes simulated by DTCManager

11 Compiled Binary Metrics

The project was compiled with `g++ -std=c++17 -Wall -Wextra -Wpedantic -g` on MSYS2 MinGW 14.2.0 (Windows). The `-g` flag embeds GDB debug symbols, which inflates the final executable size. The figures below reflect the pure operational code size per module:

Component	.text (code)	.data	.bss	Total Obj Size
Sensor	19,364 B	0 B	16 B	19,380 B
Alert	39,088 B	16 B	16 B	39,120 B
Logger	32,040 B	16 B	0 B	32,056 B

Component	.text (code)	.data	.bss	Total Obj Size
Dashboard	34,376 B	16 B	0 B	34,392 B
ThreadManager	42,244 B	0 B	0 B	42,244 B
main.cpp	103,748 B	16 B	0 B	103,764 B
tests.cpp	201,460 B	64 B	0 B	201,524 B
TOTAL (.text+.data+.bss)	472,320 B	128 B	32 B	472,480 B (~472 KB)

Table 9 — Compiled object size per component (GNU size utility). Final SmartVehicleMonitor.exe is 1.51 MB including debug symbols and linked stdlib.

12 Automotive Mapping — Real-World AUTOSAR Equivalents

The architecture was consciously designed to mirror Adaptive AUTOSAR. Every major module in this project has a direct real-world automotive equivalent:

This Project	Real Automotive Component	AUTOSAR Standard
Sensor Framework	MCAL — Microcontroller Abstraction Layer	AUTOSAR Classic
AlertManager	Vehicle Health Manager (VHM)	Adaptive AUTOSAR Platform
AlertRule (Strategy)	Fault Detection / Isolation Logic	ISO 26262 FMEA
Dashboard	Instrument Cluster / HMI ECU	Cockpit Domain Controller
EventLogger<T>	Diagnostic Event Manager (DEM)	AUTOSAR DEM SWC
DTCManager	OBD-II Fault Code Registry	ISO 15765 / SAE J2012
EdrRecorder	Event Data Recorder	ISO 15764 / EDR Standard
ThreadManager + ShutdownToken	Execution Management	ara::exec
std::mutex / lock_guard	Inter-Process Synchronisation	AUTOSAR RTE
CanBus (CanBus.hpp)	CAN Communication Stack	AUTOSAR COM / PDUR
ServiceBus (pub/sub)	SOME/IP Service-Oriented Comms	Adaptive AUTOSAR ara::com
EcuMonitor	Platform Health Management + Watchdog	ara::phm
ConfigLoader (config.txt)	Parameter Manager / ARXML Config	AUTOSAR ECUC
JsonParser.hpp	ARXML / JSON Data Exchange	Adaptive AUTOSAR DEXT
OtaSimulator	Update & Configuration Management	Adaptive AUTOSAR UCM
DriverProfile	User Preference Management	HMI / IVI Domain

Table 10 — Mapping between project components and real Adaptive AUTOSAR equivalents